

Linux, Bash, awk & sed Primer: Basics to Intermediate

A practical, copy-paste-friendly guide with core Linux commands, Bash scripting (variables, control flow, arrays, functions), and text processing using awk and sed. Includes examples and exercises from beginner to intermediate.

0) Terminal Essentials

- **Open terminal:** Ctrl+Alt+T (Linux), iTerm/Terminal (macOS), WSL (Windows)
 - **Manuals:** `man ls`, `tldr ls` (if `tldr` installed)
 - **Tab completion and history:** ↑ ↓, history, !42, Ctrl+R reverse search
-

1) Files & Navigation

```
pwd                # where am I
ls -lah            # list with sizes and hidden files
cd /path/to/dir    # change directory
mkdir -p proj/src  # create nested dirs
cp src.txt dst.txt # copy
mv old new         # move/rename
rm file            # remove file
rm -rf dir/        # remove directory recursively (danger!)
```



```
# viewing
cat file           # print file
less file          # page through
head -n 20 file    # first lines
tail -n 50 file    # last lines
tail -f logfile    # follow new lines
```

Exercise 1.1: Create notes, copy a file into it, and view only the last 10 lines.

2) Permissions & Ownership

```
ls -l              # -rwxr-x--- 1 user group size date name
chmod u+x script.sh # add execute for user
chmod 640 file      # rw for user, r for group, none for others
chown user:group file
umask              # default permission mask
```

Exercise 2.1: Make `script.sh` executable for you only; ensure others have no permissions.

3) Finding Things

```
# find files by name, type, size, or time
find . -name "*.txt" -type f
find /data -type f -size +1G
find . -type f -mtime -2    # modified in last 2 days
```

```
# combine with exec or xargs
find . -name "*.log" -print0 | xargs -0 gzip
```

```
# Locate database (needs updatedb)
locate libreadline
```

Exercise 3.1: Find all files larger than 100 MB in your home and print their sizes in human-readable form.

4) Grep & Regular Expressions

```
grep -n "pattern" file
rg pattern          # ripgrep (faster), if available
grep -E "A[CT]G{2,}" seqs.txt    # extended regex
```

```
# recursive with ignore patterns
grep -R --include='*.py' "TODO" .
```

```
# context
grep -nC2 "ERROR" app.log    # 2 lines of context
```

Exercise 4.1: Search recursively for lines containing `chr[0-9XY]+` in `*.vcf`.

5) Redirection, Pipes, and Process Substitution

```
cmd > out.txt          # stdout to file (truncate)
cmd >> out.txt          # append
cmd 2> err.txt         # stderr to file
cmd &> both.txt         # stdout + stderr to file (bash)
```

```
cmd1 | cmd2 | cmd3     # pipe output through commands
```

```
# process substitution (bash)
diff <(sort a.txt) <(sort b.txt)
```

Exercise 5.1: Combine `grep`, `cut`, and `sort -u` to list unique user names from `/etc/passwd`.

6) Archiving & Compression

```
tar -czf data.tgz data/      # create compressed archive
tar -xzf data.tgz            # extract
zip -r data.zip data/        # zip
unzip data.zip
pigz -p 8 file.fastq         # parallel gzip (if installed)
```

Exercise 6.1: Archive project/ excluding *.tmp files.

7) Networking Tools

```
curl -L -o page.html https://example.com
wget -qO- https://api.github.com | head
ssh user@server      # remote shell
scp file user@server:~/
rsync -avhP dir/ user@server:proj/
```

Exercise 7.1: Download a CSV from the web and show top 5 lines.

8) Bash Basics: Variables & Quoting

```
name="Ana"          # no spaces around =
echo "$name"         # double quotes expand vars
echo '$name'         # single quotes literal
```

```
export PATH="$HOME/bin:$PATH"
```

```
# command substitution
```

```
now=$(date +%F)
```

```
echo "Today is $now"
```

Exercise 8.1: Create a variable RUN_ID with today's date/time and use it to name a log file.

9) Bash Conditionals & Logic

```
# test numbers and strings
```

```
x=10; y=20
```

```
if (( x < y )); then echo "x<y"; fi
```

```
s="hello"
```

```
if [[ -n $s && $s == h* ]]; then echo starts-with-h; fi
```

```
# test files
```

```
if [[ -f file && -r file ]]; then echo readable file; fi
if [[ -d dir ]]; then echo directory; fi
```

Exercise 9.1: Write a snippet that exits with an error if `input.csv` does not exist or is empty.

10) Loops & Arrays

```
# for
for f in *.txt; do echo "$f"; done

# while (read lines)
while IFS=, read -r id name value; do
    echo "$id -> $name ($value)"
done < data.csv

# arrays
nums=(10 20 30)
echo "${nums[1]}"          # 20
echo "${nums[@]}"          # all elements
for n in "${nums[@]"}"; do echo "$n"; done
```

Exercise 10.1: Iterate over `*.fastq.gz`, print filename and uncompressed line count using `zcat` and `wc -l`.

11) awk — The Swiss Army Knife for Columns

Concepts: pattern { action } - Fields: \$1, \$2, ... \$NF; record separator (default newline), field separator FS (default whitespace) - Blocks: BEGIN { ... }, END { ... }

```
# Sum numbers in 3rd column
awk '{sum+=$3} END{print sum}' data.tsv

# CSV with header: mean of column "value"
awk -F, 'NR==1{for(i=1;i<=NF;i++) if($i=="value") col=i; next} {s+=$col; n++}
END{print s/n}' data.csv

# Filter: print rows where value > 10
awk -F, 'NR>1 && $3>10' data.csv

# Add computed column
awk -F, -v OFS="," 'NR==1{print $0, "zscore"; next} {print $0, ($3-50)/10}'
data.csv
```

Built-ins: `length()`, `toupper()`, `tolower()`, `substr()`, `split()`, `gsub()`

Exercise 11.1: Given `samples.tsv` (id), produce average score by group sorted descending.

Exercise 11.2: From a VCF, print `CHROM:POS REF>ALT` and INFO field AF if present.

12) sed — Stream Editor for Line/Regex Edits

Basics: address/regex + command

```
# substitution (first match per line)
sed 's/foo/bar/' file
# global substitution
sed 's/foo/bar/g' file
# in-place (make a backup on macOS: -i '')
sed -i 's/[[:space:]]\+$//' file # trim trailing spaces

# delete matching lines
grep -v '^#' file | sed '/^$/d' # or: sed -e '/^#/d' -e '/^$/d' file

# print only lines in range 10..20
sed -n '10,20p' file

# insert/append around lines matching PATTERN
sed '/^Title:/a\nDate: 2025-10-06' doc.txt # append after match
sed '/^Title:/i Author: Ana' doc.txt      # insert before match
```

Capture groups

```
# swap first and last name
sed -E 's/^([A-Za-z]+)\s+([A-Za-z]+)$/\\2, \\1/' names.txt
```

Exercise 12.1: In-place replace all tabs with 4 spaces in `*.py` under current directory.

Exercise 12.2: Remove all comment lines and blank lines from a config, writing to `clean.conf`.

13) Sorting, Uniqueness & Joins

```
sort -k3,3n data.tsv # sort by 3rd column numeric
uniq -c              # count runs (use sort first)
cut -f1,3 data.tsv   # pick columns
paste a.txt b.txt     # merge line-wise
join -1 1 -2 1 <(sort a.txt) <(sort b.txt) # join on first column
```

Exercise 13.1: From `sales.csv` (id, item, price), compute total sales per item (hint: `awk` or `csvcut` if available).

14) One-Liner Cookbook

1) Count files by extension

```
find . -type f -name '*..*' | sed -E 's@.*\.\.@@' | sort | uniq -c | sort -nr
```

2) Top 20 most common words (case-insensitive)

```
tr -cs '[:alnum:]' '\n' < book.txt | tr '[:upper:]' '[:lower:]' | sort | uniq  
-c | sort -nr | head -20
```

3) Extract 2nd and 5th columns from CSV (quote-naïve)

```
awk -F, '{print $2, $5}' data.csv | column -t
```

4) Replace spaces with underscores recursively in filenames

```
find . -depth -name '* *' -exec bash -c 'f="$1"; mv "$f" "${f// /_}"' _ {} \;
```

5) Checksum a directory

```
find dir -type f -print0 | xargs -0 sha256sum | sort -k2 > checksums.txt
```

6) Stream-join two TSVs on key (1st column)

```
join -t $'\t' <(sort -t $'\t' -k1,1 A.tsv) <(sort -t $'\t' -k1,1 B.tsv)
```

15) Cheat Sheet (Quick Reference)

- **Navigation:** ls, cd, pwd, tree
- **Files:** cp, mv, rm, mkdir, ln, touch
- **Inspect:** cat, less, head, tail, wc, du -sh *
- **Search:** grep, find, rg, locate
- **Edit:** sed, awk, cut, paste, sort, uniq, tr
- **Archive:** tar, zip, gzip/pigz, xz, zstd
- **Network:** curl, wget, ssh, scp, rsync, nc
- **Process:** ps, top/htop, kill, jobs, bg, fg, time
- **System:** df -h, free -h, uname -a, uptime

16) Further Reading

- **Bash Guide:** Greg's Wiki (mywiki.woledge.org)
 - **GNU awk Manual:** [info gawk](http://info.gawk.org)
 - **sed, awk** (O'Reilly), **The Linux Command Line** (Shotts)
-